独立按键控制LED亮灭 / 状态 / 显示二进制 / 移位

对应江协科技《51单片机入门教程-2020版》P7《[3-1]独立按键控制LED亮灭》、P8《[3-2]独立按键控制LED状态》、P9《[3-3]独立按键控制LED显示二进制》、P10《[3-4]独立按键控制LED移位》· 本章目标：读按键电平、软件消抖、状态取反、中间变量做二进制计数、位运算双向移位 · 16 题分类：一、概念填空（1-5）；二、判断改错（6-9）；三、计算（10-12）；四、应用（13-16）

一、概念填空（1–5）

1. 独立按键一脚接_____、一脚接IO口；MCU上电时所有IO口默认高电平，因此按键未按下时IO读回_____电平，按下时接通到GND读回_____电平。

答：GND（电源负极）；高（1）；低（0）。

思路：方法/口诀："按下接地读0，松开高电读1"。**步骤：**① 按键公共端接GND；② 未按时内部上拉拉高为1；③ 按下时IO与GND导通为0。

常见错法：以为按下是读高电平，把判断条件写反。

解析：寄存器不只能"写值送IO"，还能"读回IO当前电平"，这就是读按键的原理。检测到P3_1==0就说明K1被按下。

2. 本板独立按键与IO对应（板子丝印把RXD/TXD画反了）：K1=_____、K2=_____、K3=P3.2、K4=P3.3；程序里要用 P2_0 这种位名，必须包含头文件_____（reg52.h 只有 sfr 没有 sbit）。

答：P3.1（RXD）；P3.0（TXD）；regx52.h。

思路：方法/口诀："K1反接P3.1，位名要regx52"。**步骤：**① 别按常理以为K1是P3.0；② reg52.h只定义8位整体sfr；③ regx52.h才同时有sfr和sbit位声明。

常见错法：用reg52.h写P2_0，编译报"P2_0未定义"；或把K1接到P3.0。

解析：sfr定义8位寄存器整体，sbit定义其中某一位。位操作只给0/1即可，给2也按1处理（非0即真）。

3. 机械按键在按下和松开瞬间，金属弹片会抖动约_____ms；本课程用软件消抖：检测到按下后先_____ms再确认，再空转等松手，松手后再_____ms才执行动作。

答：5~10；延时20；延时20。

思路：方法/口诀："按下抖5-10ms，延时20ms两头各消一次"。**步骤：**① 单片机太快，一次按下会被识别成多次；② 检测到按下Delay(20)；③ while(P3_1==0); 等松手；④ 再Delay(20)后才动作。

常见错法：只在按下处延时一次，松手处不延时，松手抖动仍会误触发；或延时间隔选得太短没盖住抖动。

解析：硬件消抖要加触发器电路、费硬件，软件消抖只需延时，本课程统一用软件法。空循环可直接写分号while(P3_1==0);。

4. 51上电时所有IO口默认_____（写十六进制，即8个全高电平）；不能直接对P2自增或取反来做计数，必须定义一个中间变量，数据类型用_____（8位无符号、范围0~255，正好匹配寄存器位宽）。

答：0xFF；unsigned char。

思路：方法/口诀："上电全高0xFF，计数靠中间变量"。**步骤：**① P2上电=0xFF已是最大值；② 直接P2++会溢出到0x00，~P2又回0xFF，卡死；③ 定义unsigned char LED_number=0，对它自增再P2=~LED_number。

常见错法：直接P2++/P2=~P2，结果全亮或灯不动；或变量用了超范围的类型。

解析：unsigned char是8位=1字节，范围0~255，正好和8位寄存器一致。变量在=右边是"读"、左边是"写"，P2=~LED_number 只读不改LED_number。

5. 移位显示用表达式 `P2=~(0x01<<LED_number);`：0x01左移0位时只有_____灯亮；LED_number 的循环范围是_____；K2右移时减到0要跳回_____。

答：D1；0~7；7。

思路：方法/口诀："亮位1左移，0到7循环，右移0跳7"。**步骤：**① ~0x01=1111 1110即D1亮；② LED_number到8归零；③ 无符号char减到0再减会下溢到255，所以先判断==0再置7。

常见错法：右移不判断下溢，结果灯跳到255对应的乱码状态；或漏掉~（因为LED低电平点亮）。

解析：上电还要先P2=~0x01让D1默认亮，否则上电全灭、按第一下直接跳到D2。

二、判断改错（6–9）

6. 判断：读按键的 if/else 写在 main 开头一遍就行，不用放进 while(1)。

答：错误。改正：if/else 必须放进 while(1) 里面循环检测；放外面上电只判断一次，之后再也读不到按键，只有按复位键才会重新跑一遍。

思路：方法/口诀："按键判断永远套在死循环里"。**步骤：**① main执行完会从头重跑，但想实时跟手必须靠while(1)轮询；② 把if/else整段放进while(1){}。

常见错法：把读按键放在while(1)外面，现象是"上电按一下有效，之后怎么按都没反应"。

解析：单片机是不断轮询IO口的，只有把检测逻辑放进死循环，才能随时响应按键。

7. 判断：`if(P3_1=0){ … }` 就是判断 K1 是否被按下。

答：错误。改正：判断相等要写两个等号 `if(P3_1==0)`；一个等号是赋值，会把0写进P3_1，逻辑完全错误。

思路：方法/口诀："==判断相等，=赋值"。**步骤：**① 关系运算==才是比较；② =是右边算完塞给左边；③ 写错编译器虽不报错但行为全错。

常见错法：把=当==用，这类错编译器不提示，最难查。

解析：这是C语言初学者最高频笔误，P7和P8都反复强调，写完if一定先数等号个数。

8. 判断：让8个LED按二进制递增，直接写 `P2++;` 就能实现。

答：错误。改正：P2上电默认0xFF已是最大值，P2++溢出变0x00（全亮），再~P2又回0xFF，永远在两个极端间跳，看不到中间态。正解是定义 `unsigned char LED_number=0;`，动作写 `LED_number++; P2=~LED_number;`。

思路：方法/口诀："寄存器别自增，变量中转再送P2"。**步骤：**① 分析P2初值0xFF；② 中间变量从0开始数；③ 取反后写P2。

常见错法：P2++下载后全亮，还以为接反了线。

解析：寄存器是硬件状态，累加会被溢出截断；变量在RAM里自由计数，取反送出才是干净做法。

9. 判断：带参延时 `Delay(unsigned int x)` 想延100000ms就传100000。

答：错误。改正：unsigned int 范围0~65535，传超过65535会溢出绕回0，延时完全不准；另外若把Delay函数定义写在main后面，必须在main前面加分号声明函数原型，否则报"missing function-prototype"。

思路：方法/口诀："传参别过65535，后定义函数要先声明"。**步骤：**① 记unsigned int上限65535；② 超长就拆成多次调用；③ 函数放main后要加原型声明。

常见错法：传大数后延时变成一瞬间；函数没声明导致连带报一堆错。

解析：编译器报一连串错误时，双击第一个错误定位，后面的往往是连带错误。

三、计算（10–12）

10. 求值：(1) 5/2 = ？ (2) 5%2 = ？ (3) 0x01 左移3位 = ？（写十六进制） (4) ~0xFF = ？

答：5/2 = 2（整数除法丢小数）；5%2 = 1；0x01<<3 = 0x08；~0xFF = 0x00。

思路：方法/口诀："整数除丢小数，取余看整除；左移补0，按位取反0/1互换"。**步骤：**① 5/2只取整数部分2；② 5%2余1，常判整除；③ 0x01=0000 0001左移3位=0000 1000=0x08；④ ~0xFF逐位取反=0x00。

常见错法：5/2算成2.5（C里两个整数相除结果仍是整数）；~0xFF误写0x0F。

解析：位运算在单片机里高频：&全1出1、|有1出1、^不同出1、~逐位取反，<<左移高位移出低位补0。

11. 二进制计数：定义 `unsigned char LED_number=0;`，每按一次K1执行 `LED_number++; P2=~LED_number;`。问：按第1次时 LED_number=？P2=？（哪个灯亮）；从0数到255后再按，会回到几？

答：第1次 LED_number=1，P2=~1=0xFE（1111 1110），即只有D1亮；数完0~255共256个状态后再按，从255加1溢出回到 **0**。

思路：方法/口诀："变量从0自增，取反送P2，255加1绕回0"。**步骤：**① 初值0；② 第1次++得1，~1=0xFE；③ 依次00000001→00000010→…对应D1→D2…；④ unsigned char 8位，255+1溢出归0。

常见错法：以为第1次LED_number还是0；或不知道unsigned char到255会自动回卷。

解析：因为LED低电平点亮，计数亮位（bit=1）送P2前要整体取反；这正好演示8位寄存器满量程循环。

12. 移位：上电 `P2=~0x01;`，每按K1执行 `LED_number++; if(LED_number>=8)LED_number=0; P2=~(0x01<<LED_number);`。问：LED_number=3 时，0x01<<3=？P2=？；按K1多少次后LED_number从7再加一归0?

答：0x01<<3 = 0x08（0000 1000）；P2=~0x08 = 0xF7（1111 0111，D4亮）；从7再加一次到8即归零，也就是按K1第 **8** 次完成一轮回卷。

思路：方法/口诀："移3位得0x08，取反0xF7，8次一循环"。**步骤：**① 0x01左移3位=0000 1000；② 取反得1111 0111=0xF7；③ LED_number 0~7共8个位置。

常见错法：忘记~把P2写成0x08（结果只有D4灭）；或括号省掉导致优先级错。

解析：~(0x01<<LED_number) 的括号不能省；运算符优先级拿不准就加括号，既防错又好读。

四、应用（13–16）

13. 读程找错：下面这段自写程序想实现"按住K1亮D1、松开灭"，编译/逻辑都有问题，找出两处错误并改正。

```
void main(void)
{
    while(1)
    {
        if(P3_1=0)
        {
            P2_0=0;
        }
        else
        {
            P2_0=1;
        }
    }
}
```

答：两处错误——① `if(P3_1=0)` 是赋值不是判断，应写 `if(P3_1==0)`；② 程序用了位名 P3_1、P2_0 却没包含头文件，编译会报未定义，应在开头 `#include <regx52.h>`。改正：

```
#include <regx52.h>   /* 才有 sbit 位名 P3_1/P2_0 */

void main(void)
{
    while(1)
    {
        if(P3_1==0)   /* K1按下读低电平 */
        {
            P2_0=0;   /* D1亮 */
        }
        else
        {
            P2_0=1;   /* D1灭 */
        }
    }
}
```

思路：方法/口诀："先查头文件，再数等号"。**步骤：**① 用位名必含regx52.h；② if里判断相等写==；③ 整段在while(1)里轮询。

常见错法：只改等号忘加头文件；或加了头文件却仍用reg52.h。

解析：代码为自写示例（非原课程源码）。注意P必须大写，小写p是另一个标识符，51 C标识符大小写敏感。

14. 编程：按住K1时D1亮，松开K1时D1灭（不做消抖，最基础的点控）。写出完整程序。

答：

```
#include <regx52.h>

void main(void)
{
    while(1)
    {
        if(P3_1==0)   /* K1(P3.1)按下 */
        {
            P2_0=0;   /* D1(P2.0)低电平点亮 */
        }
        else
        {
            P2_0=1;   /* 松开熄灭 */
        }
    }
}
```

思路：方法/口诀："按下读0送亮，否则送灭，套死循环"。**步骤：**① 含regx52.h；② while(1)里if(P3_1==0)；③ 亮灭用P2_0位操作，不影响其它LED。

常见错法：把判断写成==0却送P2_0=1（亮灭反了）；或忘了while(1)。

解析：这是[3-1]核心效果。sbit位寄存器单独读写某一位，正好对应"只动D1不动其它灯"。代码为自写示例（非原课程源码）。

15. 编程：按一下K1、松手后D1状态取反一次（点按切换，按下不动作、松手才动作），带20ms软件消抖。写出完整程序。

答：

```
#include <regx52.h>

/* Delay(unsigned int x) 由 STC-ISP 软件延时计算器生成：
   系统频率 12.000MHz、指令集 Y1、1ms 空转段，再套 while(x){…;x--;}；
   函数体以计算器生成结果为准，此处不抄写原始空转代码。 */
void Delay(unsigned int x)
{
    while(x)
    {
        /* 计算器生成的约 1ms 空转段 */
        x--;
    }
}

void main(void)
{
    while(1)
    {
        if(P3_1==0)          /* 检测到K1按下 */
        {
            Delay(20);       /* 按下消抖 */
            while(P3_1==0);  /* 空转等松手 */
            Delay(20);       /* 松手消抖 */
            P2_0=~P2_0;      /* 松手后D1状态取反 */
        }
    }
}
```

思路：方法/口诀："按下延20、等松手、再延20、最后取反"。**步骤：**① 标准消抖模板；② while(P3_1==0); 停到松手；③ P2_0=~P2_0 对一位取反。

常见错法：在按下时就取反（变成按住反复跳）；或漏了等松手。注意全角分号会报"unprintable character"，切英文输入法。

解析：边写边测：先写P2_0=0;Delay(500);P2_0=1;Delay(500);验证1秒闪烁，再写按键逻辑。代码为自写示例（非原课程源码），延时空转体以STC-ISP生成结果为准。

16. 编程：K1按一次亮灯位左移一位、K2按一次右移一位，移到边界循环回卷，上电默认D1亮。写出完整程序。

答：

```
#include <regx52.h>

void Delay(unsigned int x)   /* STC-ISP生成的带参延时，12.000MHz/Y1 */
{
    while(x)
    {
        /* 约 1ms 空转段 */
        x--;
    }
}

unsigned char LED_number;     /* 全局变量，上电默认0 */

void main(void)
{
    P2=~0x01;                 /* 上电D1默认亮 */
    while(1)
    {
        if(P3_1==0)           /* K1=P3.1 左移 */
        {
            Delay(20);
            while(P3_1==0);
            Delay(20);
            LED_number++;
            if(LED_number>=8) LED_number=0;
            P2=~(0x01<<LED_number);
        }
        if(P3_0==0)           /* K2=P3.0 右移（与K1平行的独立if） */
        {
            Delay(20);
            while(P3_0==0);
            Delay(20);
            if(LED_number==0) LED_number=7;
            else LED_number--;
            P2=~(0x01<<LED_number);
        }
    }
}
```

思路：方法/口诀："K1加一到8归零，K2减一到0跳7，括号别省"。**步骤：**① 全局LED_number上电0；② P2=~0x01初始化；③ K1分支++并判断>=8归零；④ K2分支先判断==0再减、否则--；⑤ 统一P2=~(0x01<<LED_number)。

常见错法：K2用else if导致两键互锁；右移不判下溢；漏~或漏括号；忘写上电初始化导致按第一下跳到D2。

解析：局部变量不赋初值是随机值，移位计数要长期保持，所以用全局变量（上电默认0）。改完Create HEX要重新Rebuild才出新.hex。代码为自写示例（非原课程源码）。
